

U

O

W

CSIT305 Emerging Information Technologies and their Applications



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Text Data and Processing



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Logistics

- (1) Advanced Database Overview
- (2) Spatial Data and Computing
- (3) Graph Data and Computing
- (4) Text Data and Processing

Multi-Modal Database



- (1) AI4SE: artificial intelligence for software engineering
- (2) Software Engineering in Cloud Computing I
- (3) Software Engineering in Cloud Computing II

Software



- (1) IoT and Edge Computing
- (2) Digital Twin Technology
- (3) Blockchains and Smart Contracts
- (4) Emerging Cyber-Attacks

Networking



Outline

- Text Data and Mining
- Text Classification
- Text Clustering
- Natural Language Processing

Text Data

- Text
 - A sequence of characters with certain granularities, such as phrases, sentences, paragraphs, or a whole document. For the convenience of description, we use the document to represent a piece of text in the following.
 - Example: email, movie comment, document

Text Data

- Text data is almost always unstructured, unlike databases and data warehouses, which are structured.
- Text data is described by natural language, not purely by data, and other non-text formats such as graphics and images are not considered.
- It is normal for a document to contain tables and figures, but the main body in such documents is text. Therefore, text data mining is de facto an integrated technology of natural language processing (NLP), pattern classification, and machine learning (ML).

Text Mining

- The so-called mining usually has the meanings of “discovery, search, induction and refinement.”
- Since discovery and refinement are necessary, the target results being sought are often not obvious but hidden and concealed in the text or cannot be found and summarized in a large range.
- The adjectives “hidden” and “concealed” noted here refer not only to computer systems but also human users.

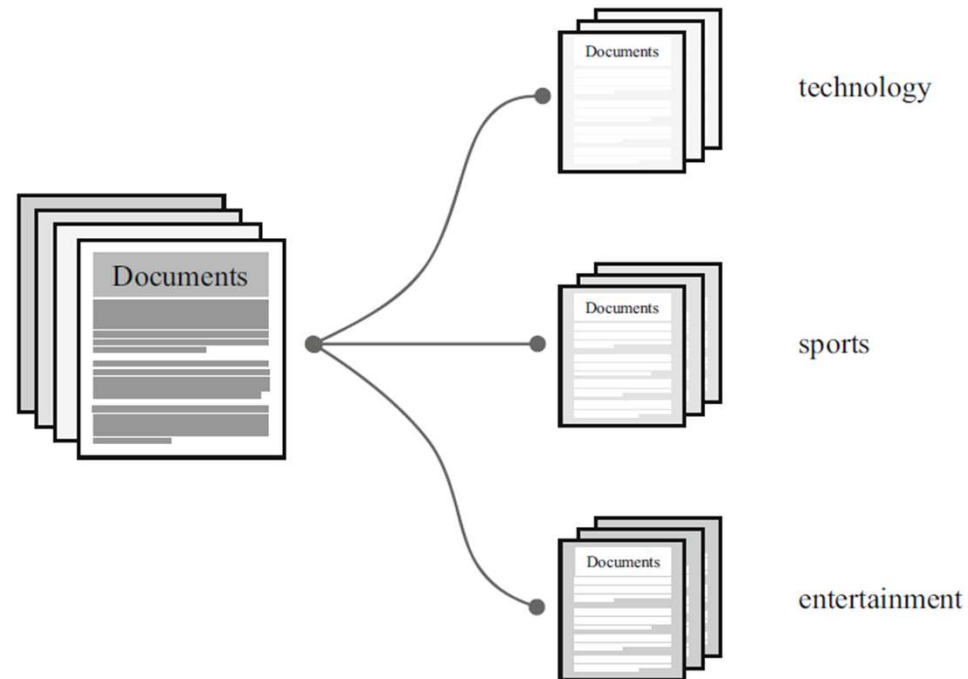
Text Classification and Clustering

- Text classification is a specific application of pattern classification technology. Its task is to divide a given text into predefined text types.
 - For a social media, the content is divided into the following categories: news, finance, sports, entertainment, cars, blog, video, house and property, etc.
 - Automatically classifying a book or an article into a certain category according to its content is a challenging task.
- Text clustering is to divide a given text set into different categories. Generally, different results can be clustered based on different perspectives.
 - Based on the text content, the text set can be clustered into news, culture and entertainment, sports or finance, and so on.
 - Based on the author's tendency, it can be grouped into positive categories (positive views with positive and supportive attitudes) and negative categories (negative views with negative and passive attitudes).

Text Classification and Clustering

- Difference between text clustering and text classification
 - Classification predefines the number of categories and the classification process automatically classifies each given text into a certain category and labels it with a category tag.
 - Clustering, by contrast, does not predefine the number of categories, and a given document set is divided into categories that can be distinguished from each other based on certain standards and evaluation indices.
- Similarities between text clustering and text classification
 - The adopted algorithms and models have intersections, such as models of text representation, distance functions, and K-means algorithms.

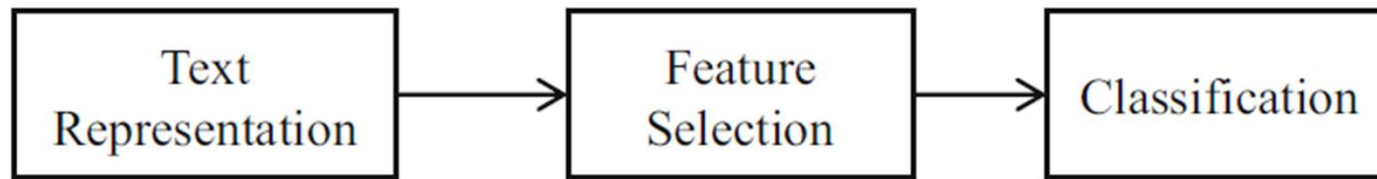
Text Classification



An example of document classification.

Text Classification

- Three stages in text classification
 - Text representation
 - Feature selection
 - Classification



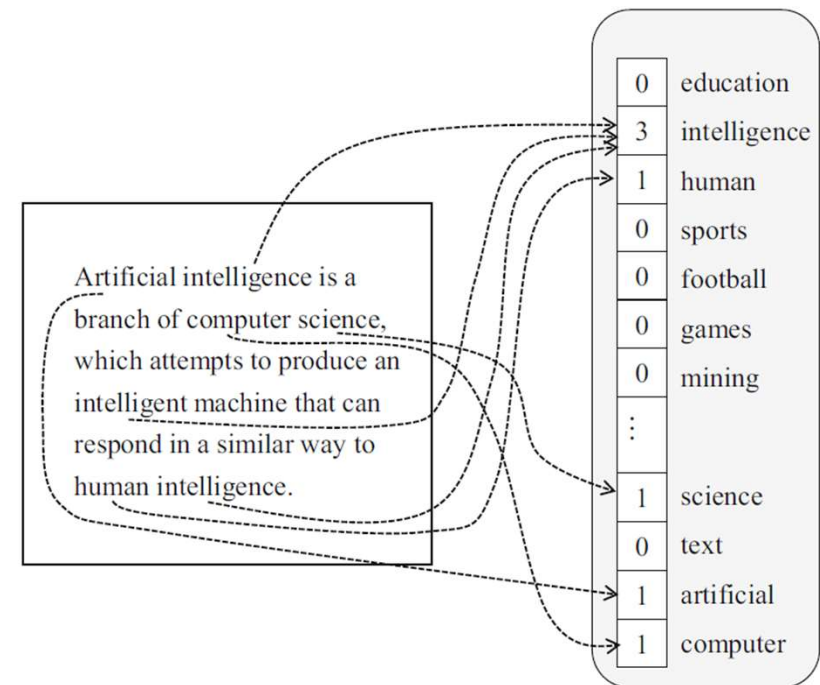
The main components of text classification based on traditional machine learning.

Text Classification

- Text representation
 - The vector space model needs a collection of terms $(t_1, t_2, \dots, t_n)$. If words are used as terms, the collection can be regarded as a vocabulary. The vocabulary can be generated from the corpus or imported from an external lexicon.
 - The terms can be viewed as a bag of words; thus, the vector space model is also called the bag-of-words (BOW) model.

Text Classification

- Use words as terms and term frequency as term weights to build a vector space model to represent the text.



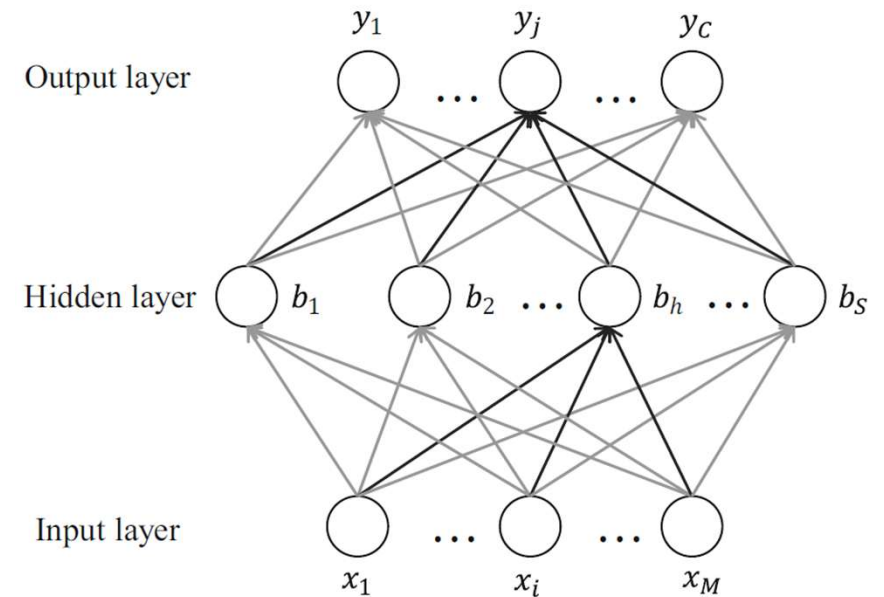
The feature weight based on feature frequency

Text Classification

- Feature selection
 - The traditional vector space model represents a document based on a high-dimensional vector space.
 - To reduce the noise contained in such a high-dimensional vector and improve the computational efficiency, it is necessary to reduce its dimension before performing classification.
- In machine learning and pattern recognition, dimension reduction methods fall into two main categories
 - Feature extraction: map the original high-dimensional sparse feature space into a low-dimensional dense feature space.
 - The classical feature extraction methods include principal component analysis (PCA) and independent component analysis (ICA).
 - Feature selection: the process of selecting a subset of features for text representation and classification.
 - In comparison with feature extraction, feature selection has been more widely discussed and used for text data.

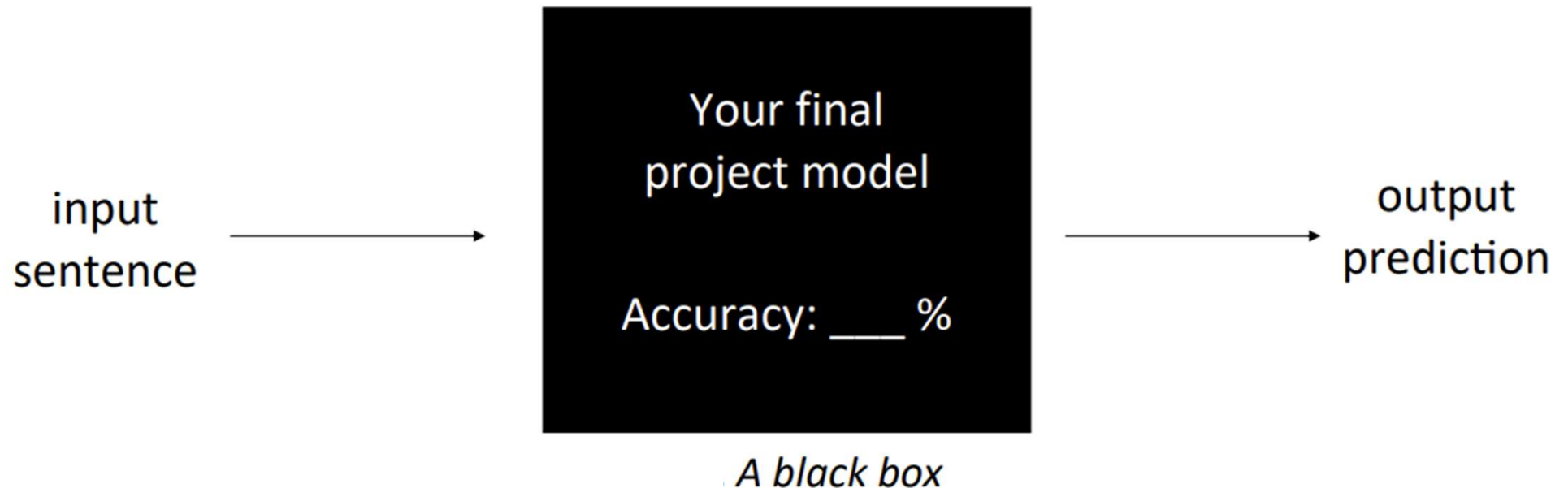
Text Classification

- Classification
 - A multilayer feed-forward neural network is a forward-structured artificial neural network that maps a set of input vectors to a set of output vectors in a multilayer fully connected manner.



The structure of a three-layer feed-forward neural network.

What's going on inside NNs?



We summarize our models with one (or a handful) of accuracies metric numbers. What do they learn? Why do they succeed and fail?

Text Clustering

- In text clustering, a cluster is represented by a collection of similar documents, and there are three main types of text similarities:
 - Similarity between two documents
 - Similarity between two document collections
 - Similarity between a document and a document collection

Text Clustering

- Text Similarity Measures
 - The similarity between documents
 - The simplest way to measure document similarity is to use the distance between two vectors in vector space.
 - **Example vocabulary:** [free, money, win, meeting]
 - Doc A: “free money win”, $A = (1, 1, 1, 0)$
 - Doc B: “free meeting”, $B = (1, 0, 0, 1)$
 - Compute distance between vectors
 - Euclidean distance : $d(A, B) = \sqrt{\sum (A_i - B_i)^2}$
 - Cosine distance: $\cos(\theta) = \frac{A \cdot B}{||A|| ||B||}$ distance = $1 - \cos(\theta)$

Text Clustering

- Text Similarity Measures
 - The similarity between clusters
 - A cluster is a collection of similar documents. The similarity between two clusters can be computed based on the similarities of the documents contained in them.
 - It can be:
 - The shortest/longest/average distance between two documents extracted from two clusters, respectively.
 - The distance between the centroid of two clusters

Text Clustering

- Text Clustering Algorithm
 - K-means clustering: divide the samples into K clusters to minimize the sum of the squared distances within each cluster.
 - Step 1: assign each sample into a cluster that minimizes the sum of the squares within clusters
 - Step 2: update the centroids for each of the K clusters
 - Step 3: perform step 1 and step 2 iteratively until the algorithm converges to a local minimum.

Text Clustering

- Hierarchical clustering is a class of cluster analysis methods that seek to build a hierarchy of clusters. It can be divided into two main types:
 - Agglomerative hierarchical clustering
 - This is a bottom-up approach where each element starts in its own cluster and similar pairs of clusters are merged as we move up the hierarchy.
 - Divisive hierarchical clustering
 - This is a top-down approach where all elements start in one cluster and splits are performed recursively as we move down the hierarchy.

NLP:Background



~50-70s

```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT 0590.DEVTO.CLIBPAU(TIMMIES) - 01.31 Columns 00001 00
Command ==>
***** Top of Data ***** Scroll ==>
000001 /* REXX EXEC *****
000002 /*
000003 /* TIMMIES FACTOR - COMPOUND INTEREST CALCULATOR
000004 /*
000005 /* AUTHOR: PAUL GABLE
000006 /* DATE: OCT 1/2007
000007 /*
000008 /*
000009 /*
000010
000011
000012 say "*****"
000013 say "Welcome Coffee drinker."
000014 say "*****"
000015 DO WHILE DATATYPE(CoffeeAmt) \= 'NUM'
000016 say ""
000017 say "What is the price of your coffee?",
000018 "(e.g. 1.58 = $1.58)"
000019 parse pull CoffeeAmt
000020 END
000021
000022 DO WHILE DATATYPE(CoffeeDk) \= 'NUM'
000023 say ""
000024 say "How many coffees a week do you have?"
000025 parse pull CoffeeDk
000026 END
000027
000028 DO WHILE DATATYPE(Rate) \= 'NUM'
000029 say ""
000030 say "What annual interest rate would you like to see on that money?",
000031 "(e.g. 8 = 8%)"
000032 parse pull Rate
000033 END
000034 Rate = Rate * 0.01 /* CHG TO DECIMAL NUMBER */
```

~80s

Can you recommend me with some restaurants in Wollongong?

Wollongong boasts a vibrant and diverse dining scene, offering everything from fresh seafood to authentic international cuisines. Here are some top-rated restaurants you might consider: visitwollongong.com.au

Balinese Spice Magic

Wollongong, NSW

A standout Indonesian restaurant known for its rich flavors and warm hospitality, offering a unique culinary experience in the heart of Wollongong. restaurantguru.com

today



Conversational Agents

- Conversational agents contain:
 - Speech recognition
 - Language analysis
 - Dialogue processing
 - Information retrieval
 - Text to speech



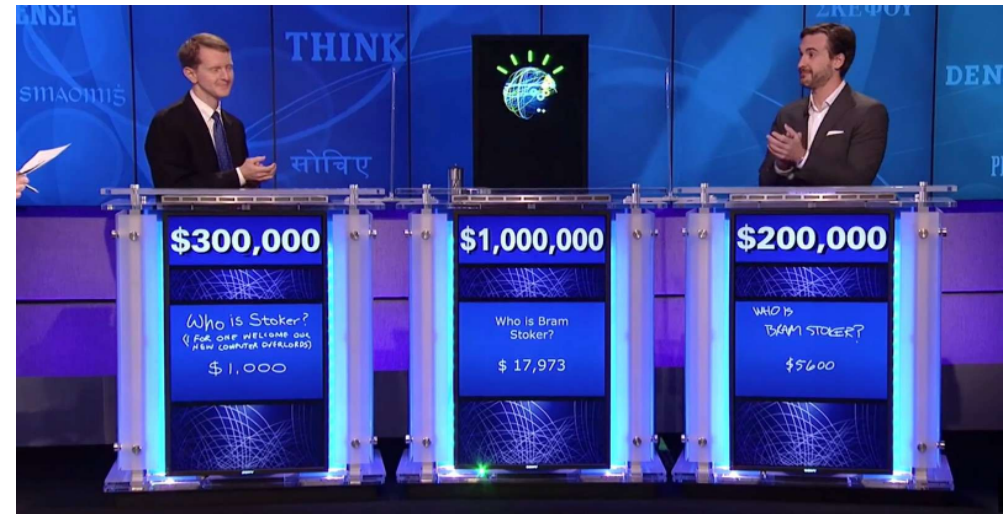
I just try to be the best me I can be

am I smart

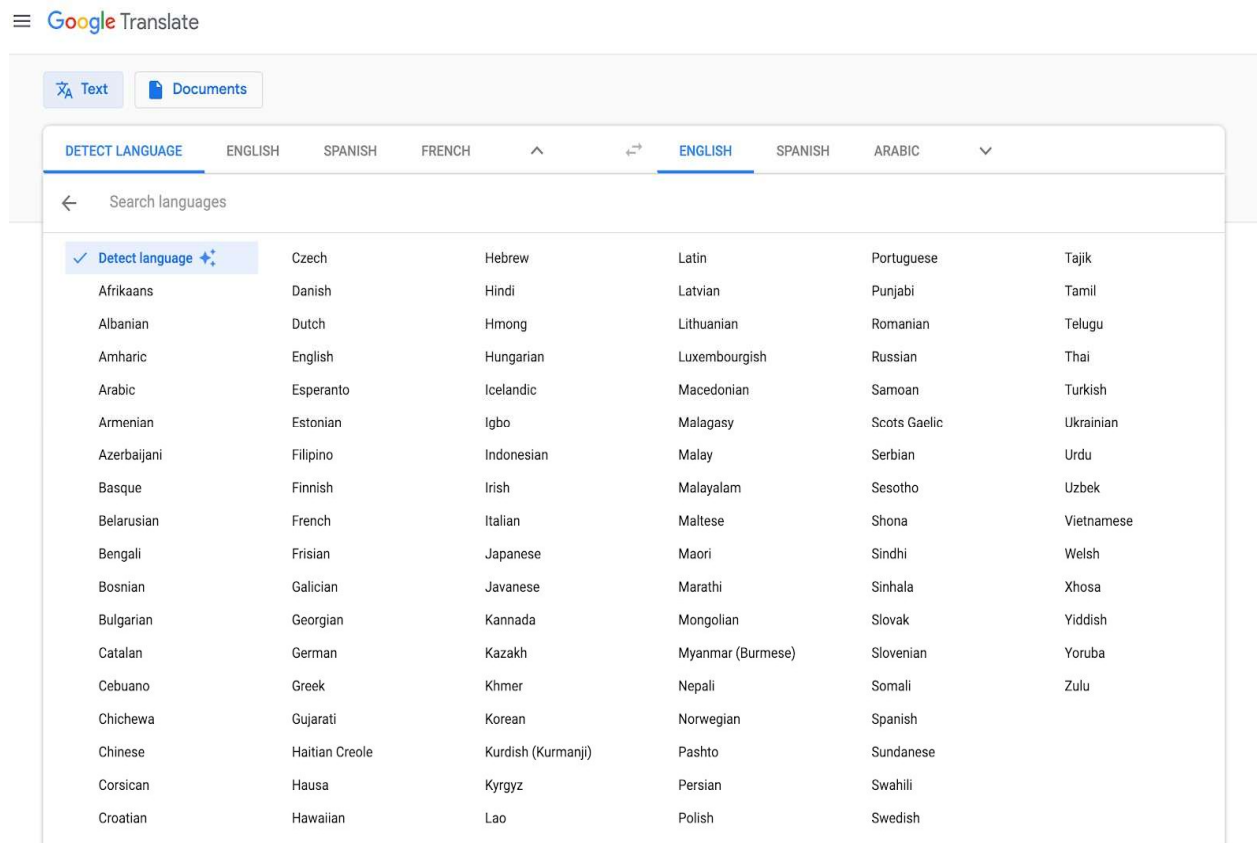
You're as smart as Grace Hopper. She
invented the first ever computer 🖥️

Question Answering

- What does “divergent” mean?
- What year was Abraham Lincoln born?
- How many states were in the United States that year?
- How much Chinese silk was exported to England in the end of the 18th century?
- What do scientists think about the ethics of human cloning?



Machine Translation



Text Summarization

- Produce a short summary of one or many longer document(s).
 - **Article:** An international team of scientists studied diet and mortality in 135,335 people between 35 and 70 years old in 18 countries, following them for an average of more than seven years. Diet information depended on self-reports, and the scientists controlled for factors including age, sex, smoking, physical activity and body mass index. The study is in The Lancet. Compared with people who ate the lowest 20 percent of carbohydrates, those who ate the highest 20 percent had a 28 percent increased risk of death. But high carbohydrate intake was not associated with cardiovascular death. ...
 - **Summary:** Researchers found that people who ate higher amounts of carbohydrates had a higher risk of dying than those who ate more fats.



Spoken Dialogue Systems -- Chatbots

- Q: Is it going to rain today?
- A: It will be mostly sunny. No rain is expected.

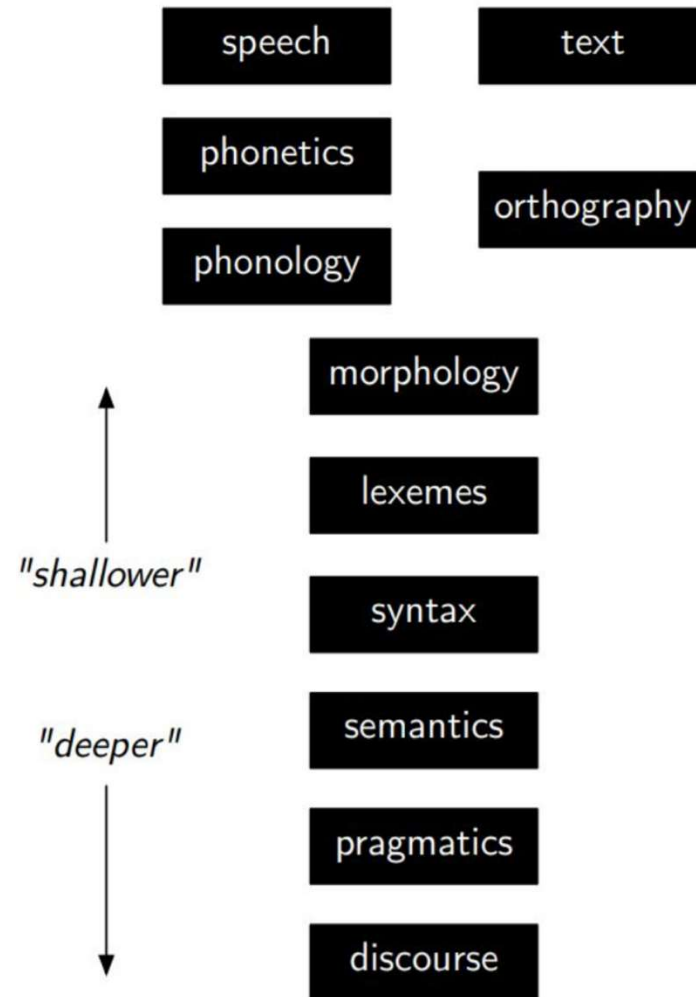


Natural Language Processing

- Applications
 - Machine Translation
 - Information Retrieval
 - Question Answering
 - Dialogue Systems
 - Information Extraction
 - Summarization
 - Sentiment Analysis
 - ...
- Core Technologies
 - Language modeling
 - Part-of-speech tagging
 - Syntactic parsing
 - Named-entity recognition
 - Word sense disambiguation
 - Semantic role labeling
 - ...

NLP lies at the intersection of computational linguistics and machine learning.

Level Of Linguistic Knowledge



Natural Language Processing (NLP) Definition

- NLP is a science that integrates linguistics, computer science, and mathematics. It studies various theories and methods for effective communication between humans and computers using natural language.
- Natural language is the language people use daily and evolves with social development. Processing refers to recognizing, analyzing, transforming, understanding, reasoning, and generating inputs of various granularities such as words, entities, sentences, chapters, topics, knowledge, etc.
- NLP aims to make machines as intelligent as humans in understanding language and to bridge the gap between human communication (natural language) and computer understanding (machine language).

Challenges

- The process focuses on various abstract concepts, semantics, and logical reasoning, which brings problems.

Some difficulties in natural language processing

Challenges	Explanation	Examples
Abstractness	Many abstract nouns are difficult to understand and express	Value dignity rights
Combinability	Natural language can combine to produce a huge amount of semantics	Hard to say, not good to say, not saying good
Evolutiveness	Natural language constantly changes with social development	Ancient and modern different meanings
Nonstandardness	Compared to code, natural language is difficult for programs to understand	If, then $\rightarrow$ if ... then ...
Subjectivity	Different people have different understandings of the same expression	Add a suitable amount of MSG

Challenges

- Ambiguity at multiple levels
 - Word senses: **bank** (finance or river ?)
 - Part of speech: **chair** (noun or verb ?)
 - Syntactic structure: **I can see a man with a telescope**
 - Multiple: **I made her duck**

Ambiguity is Ubiquitous

- Speech Recognition
 - “recognize speech” vs. “wreck a nice beach”
 - “youth in Asia” vs. “euthanasia”
- Syntactic Analysis
 - “I ate spaghetti **with** chopsticks” vs. “I ate spaghetti **with** meatballs.”
- Semantic Analysis
 - “The dog is in the **pen**.” vs. “The ink is in the **pen**.”
 - “I put the **plant** in the window” vs. “Ford put the **plant** in Mexico”
- Pragmatic Analysis
 - From “The Pink Panther Strikes Again”:
 - Clouseau: Does your dog bite?
Hotel Clerk: No.
Clouseau: [*bowing down to pet the dog*] Nice doggie.
[*Dog barks and bites Clouseau in the hand*]
Clouseau: I thought you said your dog did not bite!
Hotel Clerk: That is not my dog.



Ambiguity is Explosive

- Ambiguities compound to generate enormous numbers of possible interpretations.
- In English, a sentence ending in n prepositional phrases has *over* 2^n syntactic interpretations (cf. Catalan numbers).
 - “I saw the man with the telescope”: 2 parses
 - “I saw the man on the hill with the telescope.”: 4 parses
 - “I saw the man on the hill in Texas with the telescope”: 14 parses
 - “I saw the man on the hill in Texas with the telescope at noon.”: 42 parses
 - “I saw the man on the hill in Texas with the telescope at noon on Monday”: 132 parses



Humor and Ambiguity

- Many jokes rely on the ambiguity of language:
 - Policeman to little boy: “We are looking for a thief with a bicycle.” Little boy: “Wouldn’t you be better using your eyes.”
 - Why is the teacher wearing sun-glasses. Because the class is so bright.
 - Groucho Marx: One morning I shot an elephant in my pajamas. How he got into my pajamas, I’ll never know.
 - She criticized my apartment, so I knocked her flat.
 - Noah took all of the animals on the ark in pairs. Except the worms, they came in apples.



Challenges

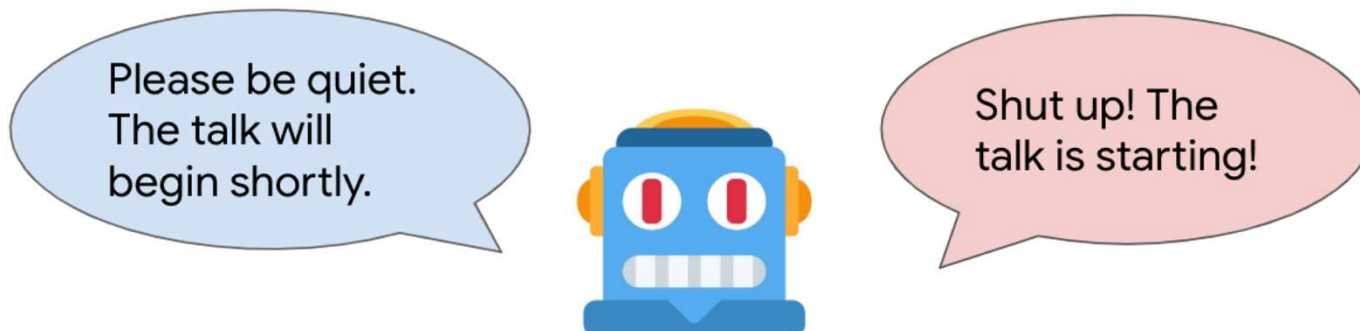
- Sparsity
 - Sparse data due to **Zipf's Law**
 - Example: the frequency of different words in a large text corpus

any word	
Frequency	Token
1,698,599	the
849,256	of
793,731	to
640,257	and
508,560	in
407,638	that
400,467	is
394,778	a
263,040	I

nouns	
Frequency	Token
124,598	European
104,325	Mr
92,195	Commission
66,781	President
62,867	Parliament
57,804	Union
53,683	report
53,547	Council
45,842	States

Challenges

- Expressivity
 - Not only can one form have different meanings (ambiguity) but the same meaning can be expressed with different forms:
 - She gave the book to Tom vs. She gave Tom the book
 - Some kids popped by vs. A few children visited
 - Is that window still open? vs. Please close the window



Desiderata for NLP Models

- Sensitivity to a wide range of phenomena and constraints in human language
- Generality across languages, modalities, genres, styles
- Strong formal guarantees (e.g., convergence, statistical efficiency, consistency)
- High accuracy when judged against expert annotations or test data
- Ethical

General-Purpose Foundation Models

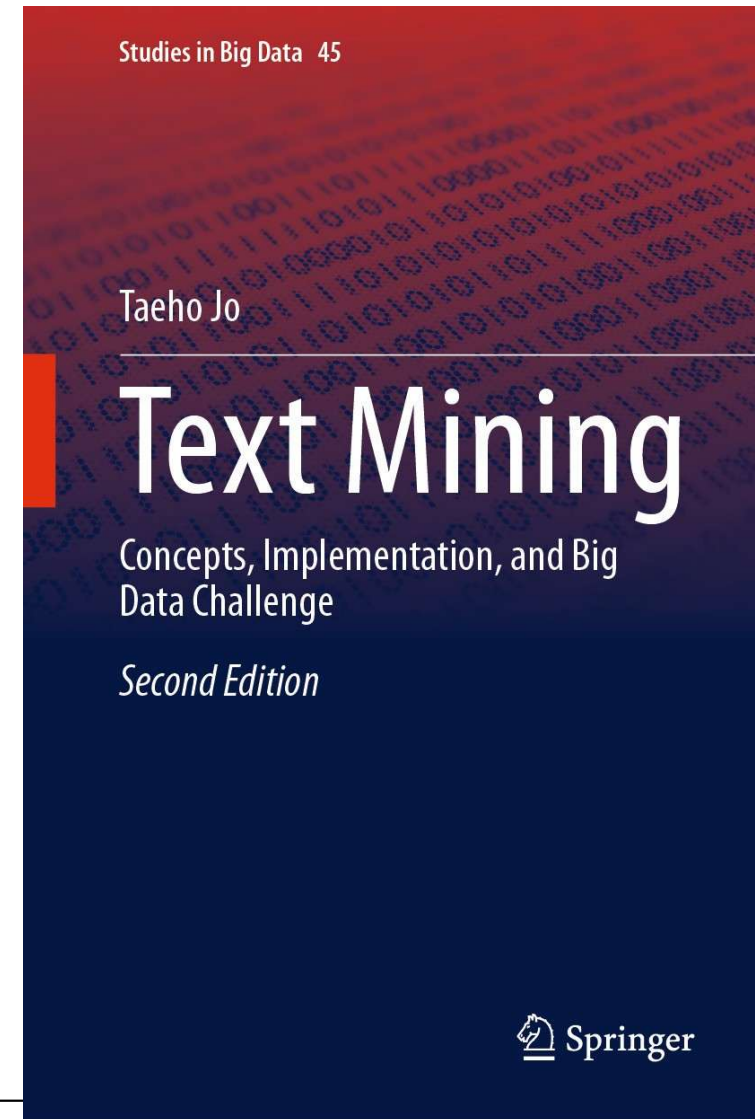
- Transformer
- OpenAI GPT models
- Google Gemini
- Anthropic Claude
- Meta Llama

NLP Applications Today

- Conversational agents
- Information extraction and question answering
- Machine translation
- Opinion and sentiment analysis
- Social media analysis
- Visual understanding
- Essay evaluation
- Mining legal, medical, or scholarly literature

Reference

- Jo, T 2025, **Text Mining: Concepts, Implementation, and Big Data Challenge**. 2nd ed., Springer, Cham.



AI4SE: Artificial Intelligence for Software Engineering



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Artificial Intelligence in Software Project

- The success of a software project highly depends on how well the project is managed. This includes crucial activities such as estimating the effort required to develop the software project, creating a software project schedule including allocation of human resources, managing project risks, monitoring progress, etc.
- Inadequate handling of such activities can thus lead to serious consequences to software companies.
- However, software project management is also a very challenging task that involves multiple business and human factors, and conflicting objectives. These challenges are exacerbated when dealing with medium- to large-size software projects.
- Under such circumstances, artificial intelligence has the potential to play a significant role in supporting software project managers, enabling them to make more informed management decisions.

Subject Logistics

- (1) Advanced Database Overview
- (2) Spatial Data and Computing
- (3) Graph Data and Computing
- (4) Text Data and Processing

Multi-Modal Database



- (1) AI4SE: artificial intelligence for software engineering
- (2) Software Engineering in Cloud Computing I
- (3) Software Engineering in Cloud Computing II

Software



- (1) IoT and Edge Computing
- (2) Digital Twin Technology
- (3) Blockchains and Smart Contracts
- (4) Emerging Cyber-Attacks

Networking



Background

- Software project management is a software engineering task concerned with ensuring that the software is delivered on time and on budget, and in accordance with the requirements of the stakeholder organisations.
- It includes crucial activities such as estimating the effort required to develop the software project, creating a software project schedule including allocation of human resources, managing project risks, monitoring progress, etc.
- Inadequate handling of such activities can lead to serious consequences to software development companies. For instance, if the software project goes over budget, the company will lose profit and may even get a negative profit margin.

Background

- Depending on how large the monetary loss is, the software company could even go bankrupt. If the software project overruns, the software development company may lose a client organisation or even be in breach of contract.
- However, software project management is also a very challenging task that involves multiple business and human factors, and conflicting objectives. These challenges are exacerbated when dealing with medium- to large-size software projects.
- Under such circumstances, Artificial Intelligence (AI) has the potential to play a significant role in supporting software project managers, enabling them to make more informed management decisions.
- How AI techniques can support software project activities?

Outline

- Software Project Scheduling
- Software Effort Estimation
- Requirement Engineering
- Code Completion with Software Mining

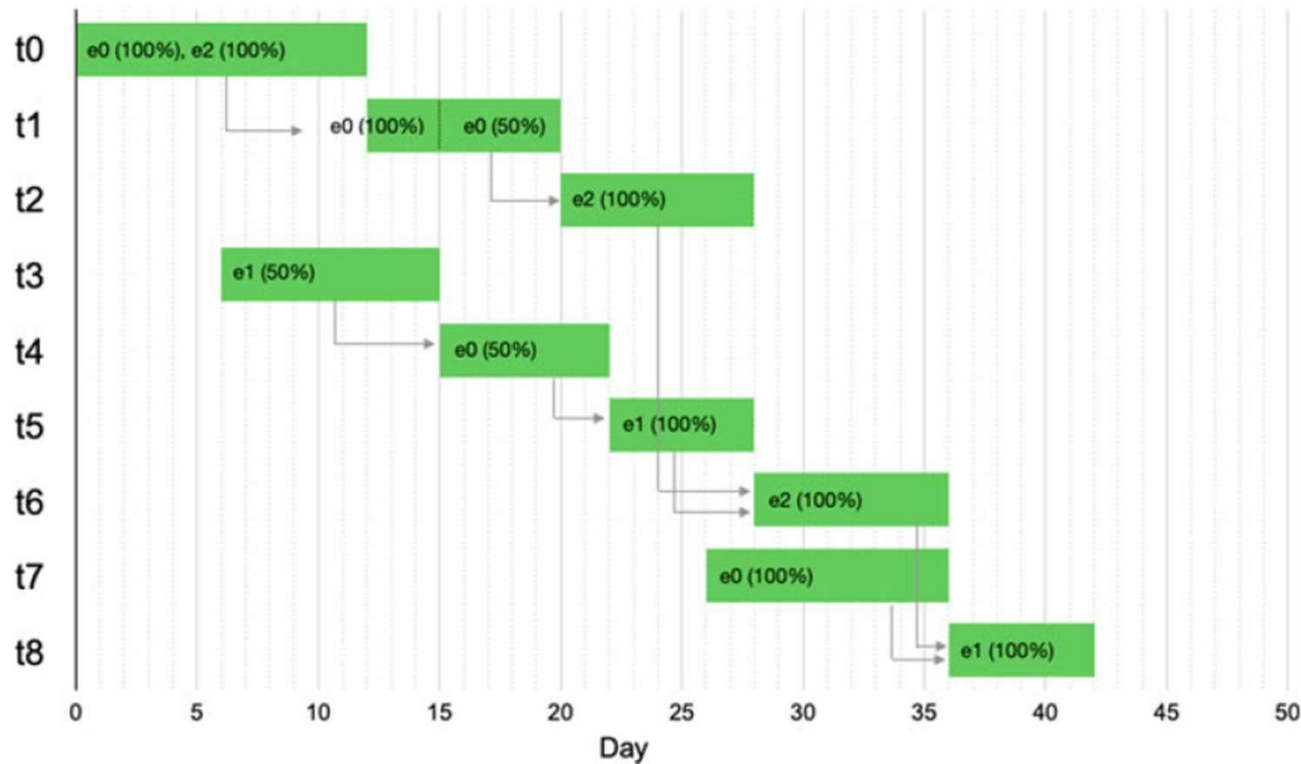
Software Project Scheduling

- Software Project Scheduling (SPS)
 - Software Project Scheduling (SPS) consists of organising the work to be done in a software project into different tasks and deciding who will work on which task and when.
 - It requires identifying the tasks to be performed in a software project; the dependencies among the tasks; the employees available to perform tasks, their skills and salaries; estimating the effort required to develop each task; allocating employees to tasks; and ultimately producing charts to communicate the project tasks, their duration and employee allocation.

Software Project Scheduling

- Software Project Scheduling (SPS)
 - In more traditional software development processes, a detailed schedule is produced at the beginning of the project, and this schedule is then adjusted as the project is developed.
 - In agile software development, the initial schedule is typically more coarse, identifying the different phases of the project.

Example



Illustrative example of project Gantt chart with nine tasks t_0 – t_8 and three employees e_0 to e_2 . Arrows represent dependencies between tasks. Percentages in brackets represent the percentage of the employee's full-time work dedicated to the corresponding task. Each vertical gray line represents the end of a given working day

Software Project Scheduling

- Typically, schedules are created with certain objectives in mind.
 - For instance, one may be interested in producing a schedule that minimises the cost and duration of the project.
 - For example, the schedule illustrated is estimated to take 42 days to complete. Possibly, this project could have taken less time to complete if more employees were allocated to it.
 - However, allocating more employees might include some employees with higher salaries, meaning that the cost of the project could potentially increase. The cost of a software project can be calculated based on the salaries of the employees allocated to it plus the cost of any other resources that may need to be used to develop the project.



Why SPS can be difficult?

- Creating a software project schedule can be a daunting task, especially when
 - The project is large and the company has a large number of employees.
 - The space of possible allocations of employees to tasks can be enormous.
 - The complexity of the allocations can also be high, given that employees can work in parallel on different tasks and each task can be allocated to more than one employee at the same time.
 - Constraints such as employees being unable to work on tasks for which they do not have the required skills, employees being unable to work more than a maximum number of hours per day, and task dependencies add to the challenge.
 - The objectives that one may be interested in optimising when creating a schedule may frequently be conflicting.



How to formulate SPS?

- The problem of finding the order with which tasks should be undertaken and the assignment of employees to tasks that minimises the duration of the software project, or the cost (salaries paid) and the amount of penalties incurred from missing milestone deadlines in the project.
- The problem of finding the allocation of employees to tasks that minimises the duration and cost (salaries paid) of the software project, while the order with which tasks are performed is calculated by a separate deterministic algorithm based on the allocations and dependencies among tasks
- Some formulations take into account different salaries for work in normal hours or overtime, the tasks' required skills and employees' skills, the level of proficiency of employees on different skills, the potential mistakes in the estimations of the effort required to complete tasks, the dynamic events that may affect the software project during its development, etc.

AI Approach for SPS

- Example – Employee Detail

Table 2.1 Example of employees available for a given project

Employee name	Hourly salary	Skills	Normal hours	Maximum dedication
$e_0 = \text{John}$	$s_0 = \$10$	$sk_0 = \{ \text{Python, SQL} \}$	$h_0 = 8 \text{ h}$	$md_0 = 1$
$e_1 = \text{Tom}$	$s_1 = \$30$	$sk_1 = \{ \text{C++}, \text{TCP/IP}, \text{HTTP} \}$	$h_1 = 8 \text{ h}$	$md_1 = 1$
$e_2 = \text{Jack}$	$s_2 = \$25$	$sk_2 = \{ \text{Java}, \text{Javascript}, \text{SQL}, \text{JSON}, \text{Testing} \}$	$h_2 = 8 \text{ h}$	$md_2 = 1$
$e_3 = \text{Claire}$	$s_3 = \$27$	$sk_3 = \{ \text{UML}, \text{Java}, \text{SQL}, \text{Testing}, \text{Mocks} \}$	$h_3 = 8 \text{ h}$	$md_3 = 1$
$e_4 = \text{Mary}$	$s_4 = \$35$	$sk_4 = \{ \text{Data science}, \text{Python}, \text{SQL}, \text{JSON} \}$	$h_4 = 8 \text{ h}$	$md_4 = 1$
$e_5 = \text{Junior}$	$s_5 = \$20$	$sk_5 = \{ \text{C++}, \text{TCP/IP}, \text{HTTP} \}$	$h_4 = 8 \text{ h}$	$md_5 = 1$

AI Approach for SPS

- Example – Task Precedence Graph

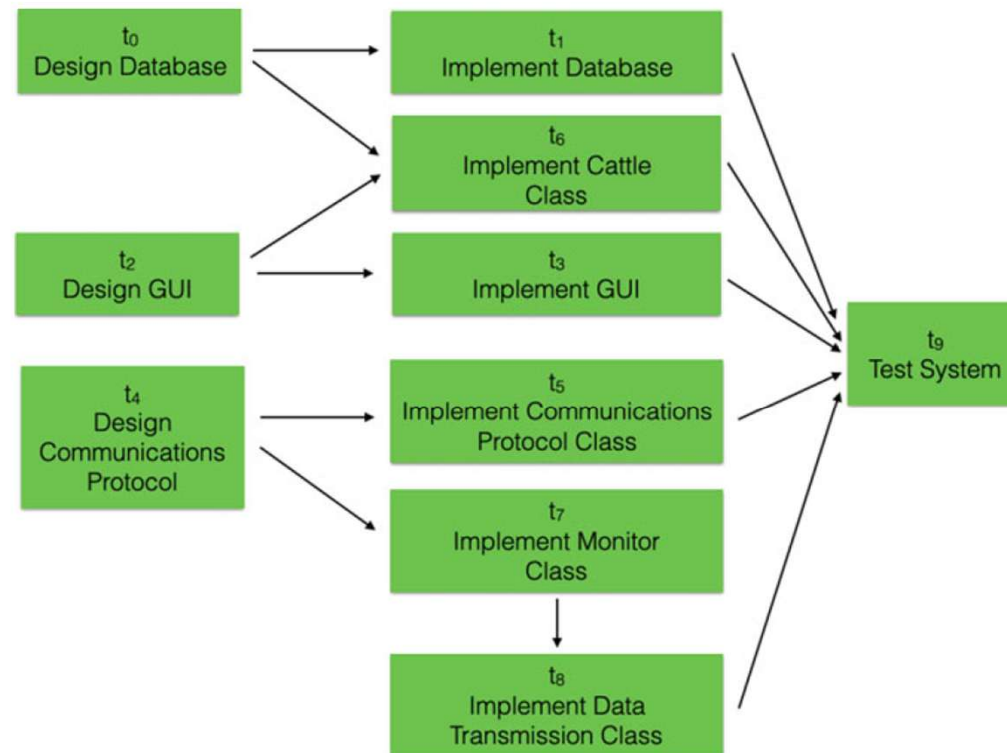


Fig. 2.2 Example of task precedence graph for a cattle monitoring and tracking project

AI Approach for SPS

- Evolutionary Algorithms
 - Evolutionary algorithms are computer methods that try to solve problems by mimicking how nature works—just like how animals evolve over time.
 - Start with random guesses. The algorithm begins with a group of random solutions to the problem. You can think of each solution as a "candidate" or "individual."
 - Check how good each solution is. Each candidate is tested to see how well it solves the problem. This is called evaluating its "fitness."
 - Pick the better ones. The best-performing candidates are selected to be "parents" for the next generation—just like how the fittest animals are more likely to survive and reproduce.
 - Create new solutions. New candidates are made by mixing parts of the parent solutions (this is called crossover) and by making small random changes (called mutation).
 - Repeat the process. The new solutions replace the old ones, and the process is repeated over many generations.
 - Stop when good enough. The algorithm keeps going until it finds a good solution or reaches a set number of rounds.



AI Approach for SPS

- Example – A possible solution

Table 2.3 Example of solution for scheduling a cattle monitoring and tracking project. The numbers in each cell $[i, j]$ correspond to the percentage of employee e_i 's normal working time dedicated to task t_j

	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9
e_0	1	1	0	0	0	0	0	0	0	0
e_1	0	0	0	0	0	0	0	0	0	0
e_2	0	0	1	0.25	0	0	0.5	0	0	1
e_3	0	0	0	0	0	0	0	0	0	1
e_4	0	0	0	0	0	0	0	0	0	0
e_5	0	0	0	0	1	0.5	0	0.5	1	0

AI Approach for SPS

- Example – Gantt Chart

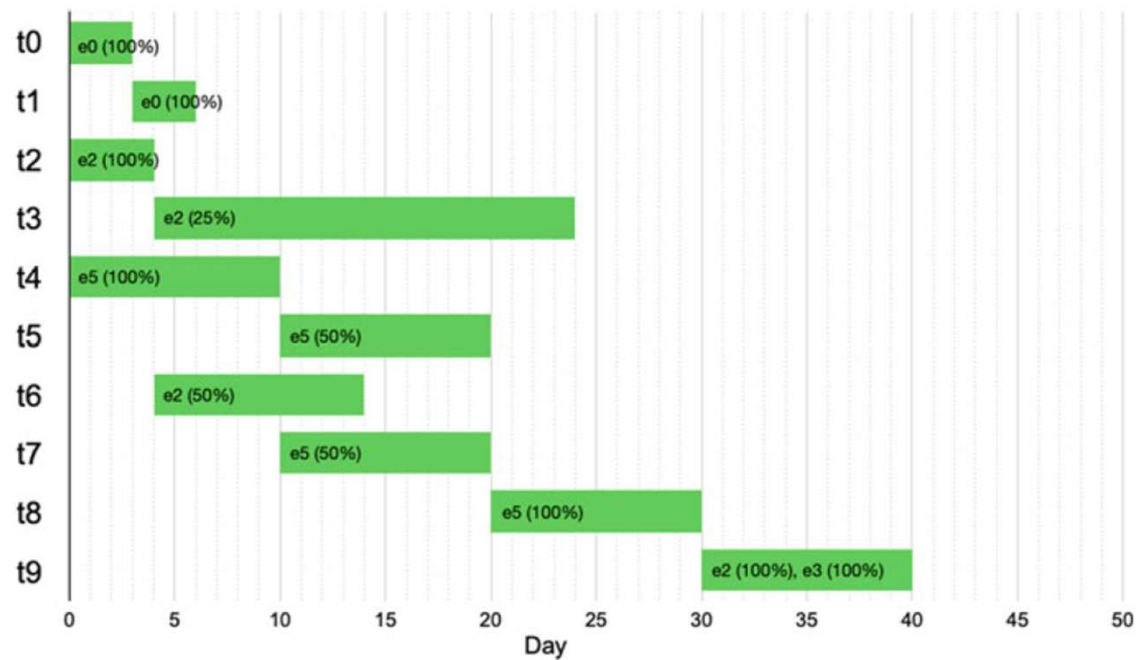


Fig. 2.3 Gantt Chart for the Solution from Table 2.3. Task dependencies are illustrated in Fig. 2.2 and are omitted here so that the Gantt Chart is easier to read. The duration of the project is 40 days and the project cost is \$12,240 based on this solution

Software Effort Estimation

- Software Effort Estimation (SEE) is the process of predicting the effort (e.g. in person-months) required to develop a software project. It is typically the first step of the software development process and can contribute to a successful software project management, constituting the basis for the subsequent steps such as planning, estimating cost, managing and reducing project risks.
- When estimating the effort of a software project as a whole in the traditional waterfall-like software development, software requirements of the customers are elicited at the beginning of the project and typically remain the same or do not change much throughout the software development process.

Software Effort Estimation

- Therefore, SEE often takes place in the early stages of software development, and based on it project managers make important decisions such as the budget, the bidding price and the subsequent planning and control.
- The estimation is typically made based on information regarding the project and the team intended to work on the project, such as functional size (a metric estimating the size of the software to be developed), software development type (new development, enhancement, re-development) and development team skills (low, medium, high)

Software Effort Estimation

- When adopting agile software development processes, software is typically developed incrementally in iterations, catering for feedback and requirements from the customers.
- Specifically, at the end of each developing iteration, developers would release the software at the current status to the customer requiring for feedback such as new functionalities and changes to implemented functionalities.
- Developers and customers then negotiate and agree on the requirements to be developed in the next iteration. In this scenario, the effort for each iteration (not for the remaining iterations of the project) is typically estimated right before starting to develop this iteration.

Software Effort Estimation

- Therefore, SEE often takes place multiple times throughout the entire process. Similar to the traditional development process, the estimation of an agile project to be developed is typically made based on information regarding the project's iteration and the team assigned to work it, such as task size in this iteration, development team's skills (low, medium, high) and development team's experience (with/without prior experience on the task).

Software Effort Estimation

- Good effort estimation is important for software project management. Both over and under-estimation can cause serious problems to the organisation.
 - Over-estimation may result in a company losing bids for contracts or wasting resources.
 - Underestimation may lead to poor product quality, unsatisfied customers, delayed or even incomplete software systems.
- For example, NASA had to shut down its incomplete Checkout & Launch Control System project after it massively exceeded the budget. Nevertheless, providing accurate effort estimations is very challenging.
- Project managers have mainly relied on expert judgement to produce estimations. As a result, the estimation quality is highly affected by the experience and the capability of the experts. Such expert-based estimation typically suffers from a number of issues: they can be costly, the prediction process is not explicit, they are not repeatable, they can be influenced by irrelevant factors such as being sensitive to political pressure, and they can have personal bias

Software Effort Estimation

- Artificial Intelligence (AI) can be used for SEE as a decision support tool. Based on estimations provided by AI models, software project managers can potentially justify, criticise or adjust the estimation derived by the experts. Such model-based estimation can potentially circumvent problems faced by the experts: they are repeatable, objective, efficient and can often provide better understanding of the estimation process.
- Therefore, AI approaches could potentially provide a more justifiable effort estimation than those decided based on “gut feeling”. They could also be used to investigate ‘what-if’ scenarios, where required efforts when using different development resources, different development teams and so on could be compared to each other, enabling practitioners to make more informed planning decisions.

Software Effort Estimation

- To build an AI approach for SEE, one needs to
 - Extract data features describing previously completed software projects (or agile iterations) and their project team(s), and to collect the actual effort spent to fulfil these projects. This can form a training set, based on which an SEE model can be constructed, following a machine learning algorithm.
 - After that, software project managers can use the SEE model to estimate the effort to develop a new software project (or agile iteration).

Software Effort Estimation

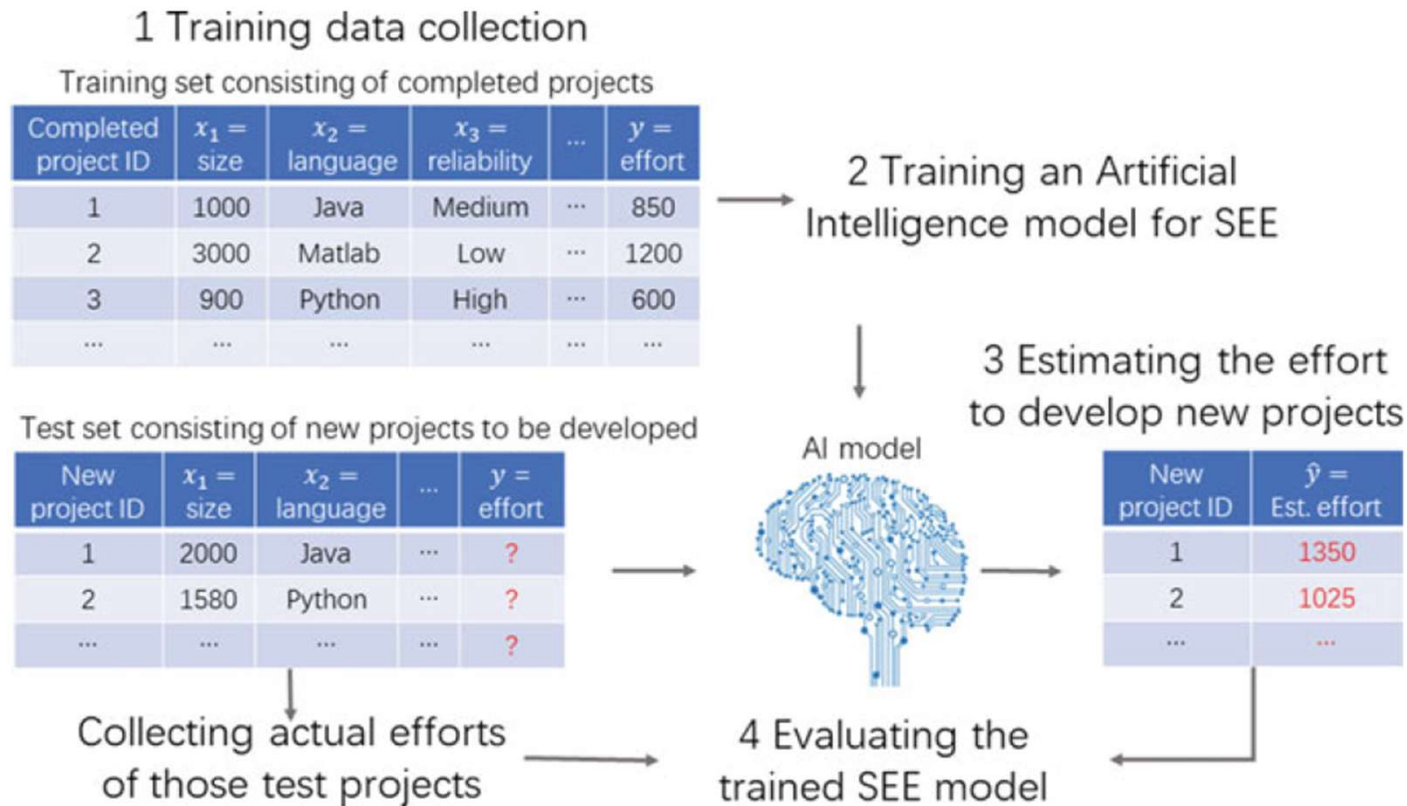


Fig. 2.9 Typical procedures of SEE from the AI viewpoint. Model adaptation is not illustrated here

Requirement Engineering

- Requirements Engineering (RE) aims at understanding the purpose of a software system to be built, and at keeping the whole development project aligned with it. RE's core question can be stated as What's the right system to be built?, while the overall purpose of software engineering concerns how to build such a system in the right way.
- RE tasks are usually performed at the beginning of each iteration in a software system development project, but are also directly linked to specific software engineering tasks, such as testing.
- Many decisions in software development projects are affected by how well RE is performed. Most failures of a software project are due to errors introduced during requirements engineering activities.

Requirement

- Requirements of a software system can be defined as the specification of how the system should behave, or of a system property or attribute.
- The basis for building good quality requirements specification is to understand the application domain, also called the world by Zave and Jackson in their formulation of the requirement problem, and to differentiate domain knowledge from the artefacts, such as software design, and technological platforms that we use to build the system-to-be, which is called the machine domain.

Requirement

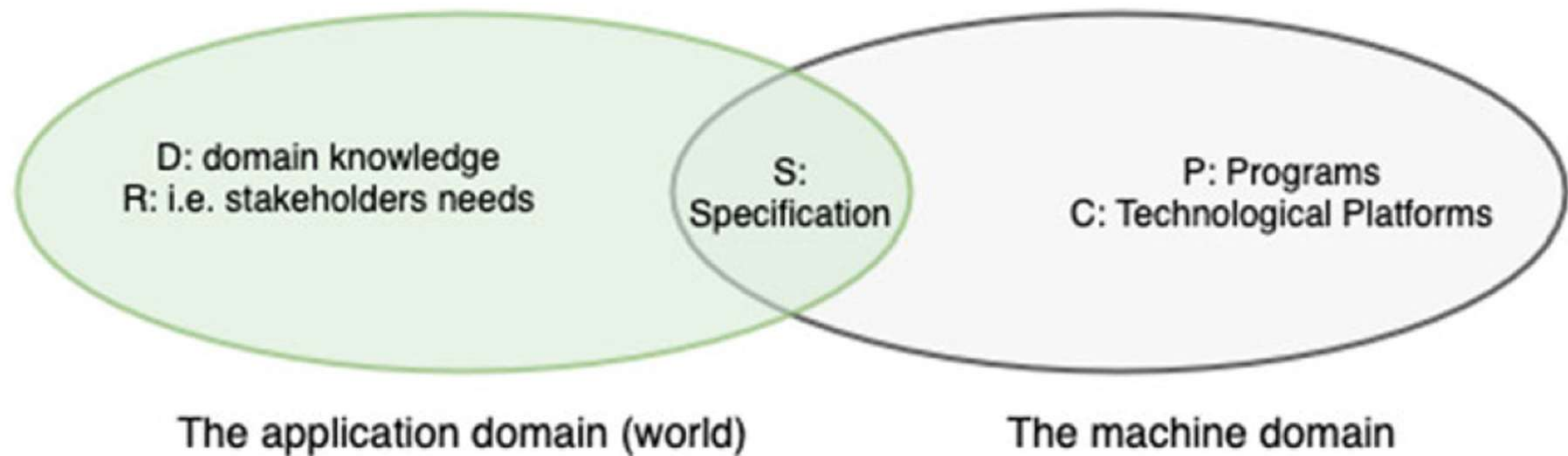


Fig. 3.1 The Requirements Problem according to Zave and Jackson [27]

Requirement

- Requirements can be differentiated depending on the stakeholder's perspective.
 - Business requirements correspond to the goals of the organisation that builds a software product.
 - User requirements correspond to goals and tasks that users want to be able to perform with the system to be built, or to system qualities the users expect.
- Software requirements are also differentiated depending on what they describe.
 - Functional requirements describe the behaviour that the software system must exhibit under specific conditions.
 - Non-functional requirements describe qualities that characterise such behaviours as performance and scalability, or properties that software systems must exhibit, or still constraints that it has to meet, e.g. portability.

Requirement

- Requirements engineering offers a set of methods and techniques for managing requirements.
- These methods are used by requirements engineers to perform RE tasks in each specific phase of the requirements process, which in turn involves a set of iterations over four main phases.

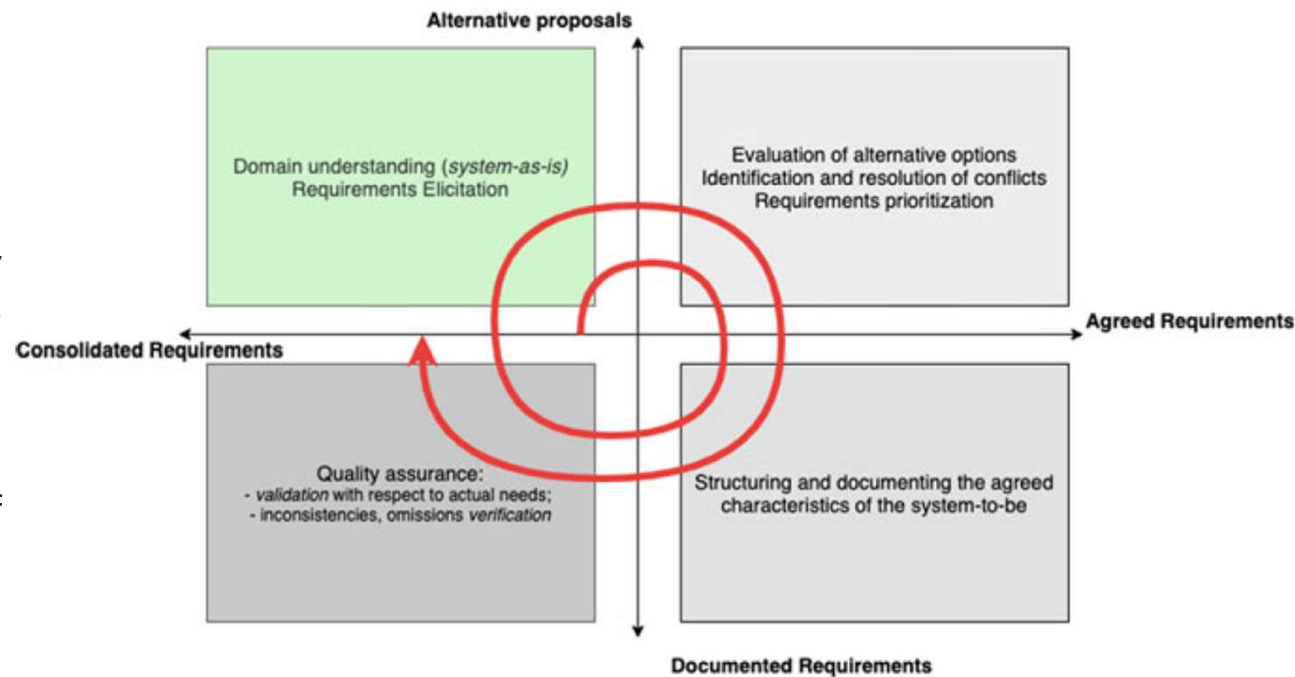


Fig. 3.2 Requirements Engineering life cycle. Adapted from [25]

Requirement Elicitation

- Requirements elicitation is an important phase of RE where the analyst tries to capture the requirements for the system to be built from the involve stakeholders.
 - The stakeholders could be the clients, end users, domain experts, investors, etc.
- As a result, understanding the needs of the stakeholders and capturing them in a set of clearly defined requirements is typically a challenging task.
- The task is made more difficult by the fact that the requirements analyst usually does not have sufficient knowledge about the domain of the system being built.
- Hence, the analyst cannot be sure to have elicited all requirements. All these factors make requirements elicitation a deceptively easy and straightforward but actually difficult task.



Requirement Elicitation

- With the proliferation of the Internet and mobile applications as well as online platforms that support end user discussions, there is increasing feedback from end users regarding a specific system (software) they are using.
- Hence, in the context of software evolution, the requirements analyst has an additional source of information that comes directly from the end users of the system. Depending on the type of software, the platforms could be different.
- For instance, large and widely used software have dedicated online platforms where end users discuss potential problems they faced while using the software. There are also systems that have issue tracking systems that are directly available to end users so that they can report issues directly via the tracking system. Additionally, most mobile applications provide a mechanism for end users to submit feedback about the application in the form of user reviews. Such reviews, typically containing short textual messages and ratings/stars, are facilitated by the application marketplaces that host the applications, e.g., the Google Play Store or the Apple App Store.

Requirement Elicitation

- A natural consequence of this abundant availability of end user feedback is to see how it could be useful for the purpose of eliciting new requirements for a given system. This topic has been the focus of growing research efforts and a number of interesting approaches have been proposed in order to make use of end user feedback as a source of new requirements.

Requirement Elicitation

- User feedback is expressed in natural language and predominantly in a textual manner. Hence, techniques for dealing with natural language content are suitable for the task. However, NLP techniques are generic and work at a lower level of granularity with respect to the conceptual elements one would like to extract for the purpose of requirements elicitation.
- For this purpose, additional concepts concerning natural language are typically introduced for processing user feedback and eventually extracting relevant information at a higher level of abstraction. One such approach is speech acts analysis.
 - Speech acts capture what a person is trying to communicate when he/she addresses another person. For example, if a user writes “I would like to be able to save a file to PDF.”, the user is communicating the desire to have a specific thing (requestive speech-act).

Requirement Elicitation

- Speech acts, identified from the user feedback messages, form the foundation on which the process of extracting requirements relevant information is built.
- Given a set of user feedback messages, the overall process is as follows:
 - Prepare the (raw) feedback messages by cleaning the text, and applying NLP-based low-level filtering operations, e.g., removing special characters.
 - Identify the speech-act(s) present in each feedback message.
 - Based on the identified speech acts, determine whether or not that particular feedback message is relevant for requirements elicitation or not.

Requirement Elicitation

- Step 1 is implemented by applying pre-processing techniques to the text via pattern matching.
- Step 2 is implemented by means of a set of rules that can identify one or more speech acts in the given text. Each speech-act category has its own rules that determine whether or not it is present in a given text.
- Step 3 is implemented by training a ML classifier, which is based on the identified speech acts, that is able to categorise the feedback message into a predefined set of outcomes.

Code Completion

- A code completion (CC) tool helps a developer to fill in the code under editing. The granularity of the code to be completed could be at the token, statement, or code fragment level.
- Code completion, commonly used by software developers, is the third most important feature in an IDE (besides editing and compiling).
- CC helps speed up the process of coding by filling in the desired code and reducing common mistakes. Programmers use CC as often as using the copy and paste feature. In all major IDEs (e.g., Eclipse and IntelliJ IDEA), the CC feature has evolved from basic support to more advanced services.

Code Completion

- Basic code completion helps a developer complete the names of classes, methods, fields, and keywords within the visibility scope.
- When code completion is invoked, the tool analyzes the context and suggests the choices that are reachable from the current position. If CC is applied to a part of a field, parameter, or variable declaration, the tool suggests a list of possible names depending on the entity's type.
 - For example, if a variable *v* of `String` is typed, an IDE will recommend the list of possible fields and methods of the type `String`. In general, the list of those accessible elements is sorted in alphabetic order.

Code Completion

- More advanced code completion services, such as IntelliJ, include the completion of common program constructs:
 - the right part of assignment statements
 - variable initializers
 - return statements
 - the list of arguments of a method call
 - after the new keyword in an object declaration
 - in chained expressions
 - a method declaration
 - a code construct, e.g., punctuation in a condition statement
 - wrapping a method call of an argument.

Code Completion

- The key issues with the program analysis (PA) approaches that have been hindering the effectiveness of CC have two folds.
 - The **first** issue with PA-based CC approaches is that there are a large amount of possibilities with equal likelihoods of code candidates that can be filled in at the cursor. Although the number of valid candidates for the next token is limited, the number of possible valid (complete) statements at the suggestion point might be combinatorially explosive or even infinite.
 - For example, after “*len* =”, the valid next-token candidates include a limited set of code tokens including the appropriate prefix operators (e.g., “++” and “–”), the open parenthesis, field access, method call, and local variable (e.g., *children* and *f*). However, there is potentially an infinite number of valid statements at that suggestion location.

Code Completion

- The key issues with the program analysis (PA) approaches that have been hindering the effectiveness of CC have two folds.
 - The **second** issue is the long and unranked candidate list of candidates due to the inability of PA to evaluate their occurrence likelihoods. In brief, program analysis could produce a large number of candidates with equal occurrence likelihoods, despite that the candidates are syntactically or semantically valid.

Code Completion with Software Mining

- API Code Recommendation Using Statistical Learning from Fine-Grained Changes
 - APIRec is an API recommendation approach that taps into the predictive power of repetitive code changes to provide relevant API recommendations for developers. APIRec is based on statistical learning from fine-grained code changes and from the context in which those changes were made. The intuition is that when developers make low-level changes, even non-contiguous changes are connected. These connections exist because the developer made the changes with a higher level intent in mind (e.g., adding a loop collector). Grouping fine-grained changes by higher level intent allows us to cut through the noise of unrelated tokens that may surround the recommendation point.
 - To find these groups of fine-grained changes, statistical learning is used in APIRec on a large code change corpus. The changes that belong to higher level intents will co-occur more frequently than non-related changes.



Code Completion with Software Mining

- API Code Recommendation Using Statistical Learning from Fine-Grained Changes
 - APIRec computes the most likely API call to be inserted at the requested location where an API call would be valid. It works in three steps:
 - it builds a corpus of fine-grained code changes from a training set
 - it statistically learns which fine-grained changes co-occur
 - it computes and then recommends a new API call at a given location based on the current context and previous changes.

AI-based Code Completion

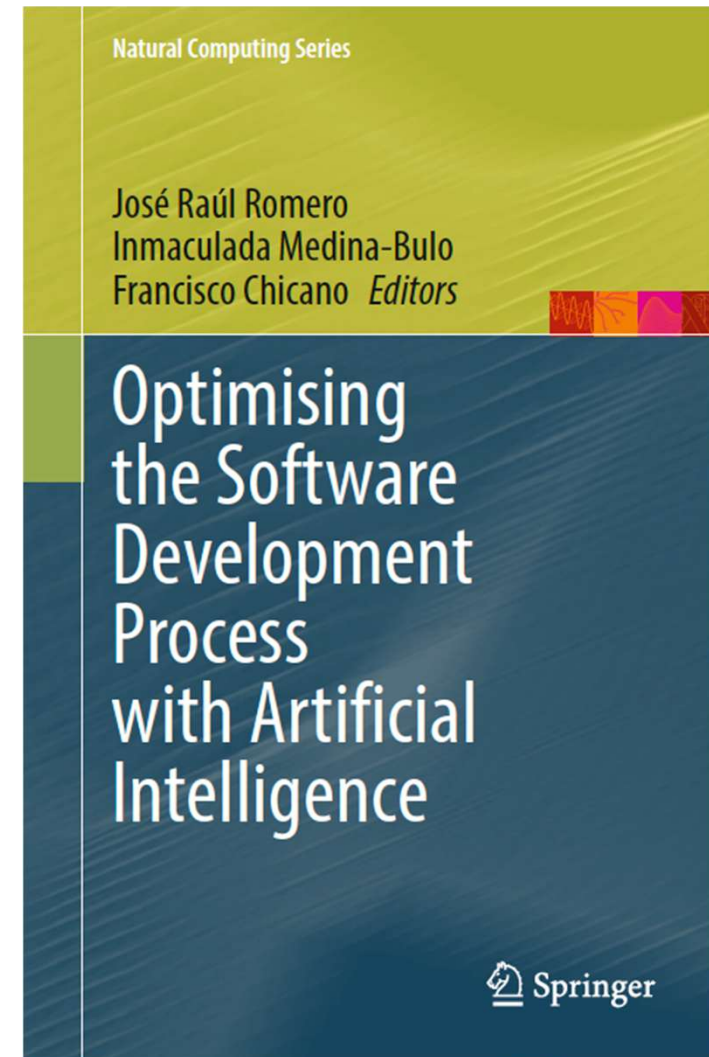
- AI-based code completion is a software development tool that uses machine learning models (usually large language models) to predict and generate code based on the context of what a developer is writing.
 - Collects coding context
 - Encodes the context using a Transformer model
 - Uses patterns learned from large-scale training data
 - Predicts the next likely code tokens
 - Generates multiple candidate completions
 - Ranks and selects the best suggestion
 - Outputs inline suggestions to the developer
 - Continuously updates as you type

Example

- Discover Amazon Q: AWS's Innovative Generative AI Assistant | Amazon Web Services
- <https://www.youtube.com/watch?v=fQITLL5WncE>

Reference

- Romero, JR, Medina-Bulo, I & Chicano, F 2023, **Optimising the Software Development Process with Artificial Intelligence** 1st ed. 2023., Springer Nature Singapore, Singapore.



U

Thanks

O



W

UNIVERSITY
OF WOLLONGONG
AUSTRALIA